

CHAPTER 03

OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

3장 OpenCV 주요 클래스

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

3.1 기본 자료형 클래스

기본 자료형 클래스

- OpenCV는 컴퓨터 비전 프로그래밍에서 자주 사용되는 다양한 클래스를 지원함
- Point_ 클래스 : 픽셀의 좌표를 저장하는 클래스
- Size_ 클래스 : 사각형 영역의 크기를 저장하는 클래스
- Rect_ 클래스 : 사각형의 위치와 크기 정보를 저장하는 클래스
- RotatedRect 클래스 : 회전된 사각형을 표현하는 클래스
- Range 클래스 : 범위 또는 구간을 저장하는 클래스
- String 클래스 : 문자열을 저장하는 클래스

Point_ 클래스

- 2차원 평면 위에 있는 점의 좌표를 표현하는 템플릿 클래스임
- 2차원 좌표를 저장하는 x와 y라는 이름의 멤버 변수를 가지고 있음

코드 3-1 간략화한 Point_ 클래스 정의와 이름 재정의

```

01  template<typename _Tp> class Point_
02  {
03  public:
04      Point_();
05      Point_(_Tp _x, _Tp _y);
06      Point_(const Point_& pt);
07
08      Point_& operator = (const Point_& pt);
09
10      _Tp dot(const Point_& pt) const;
11      double ddot(const Point_& pt) const;
12      double cross(const Point_& pt) const;
13      bool inside(const Rect_<_Tp>& r) const;
14      ...

```

템플릿 매개변수

Point_ 클래스

코드 3-1 간략화한 Point_ 클래스 정의와 이름 재정의

```

16     _Tp x, y;
17 };
18
19 typedef Point_<int>    Point2i;
20 typedef Point_<int64> Point2l;
21 typedef Point_<float> Point2f;
22 typedef Point_<double> Point2d;
23 typedef Point2i       Point;
  
```

x축 좌표, y축 좌표



Point_ 클래스

- `Point_` 클래스는 템플릿 클래스 -> 객체 선언시 사용할 자료형을 명시해야 함

`Point_<int> pt;`

- 자주 사용하는 자료형에 대하여 `Point_` 클래스 이름을 재정의하여 제공함
 - `Point2i` 클래스 : 점의 좌표가 int 형일때 사용함
 - `Point2f` 클래스 : 점의 좌표가 float 형일때 사용함
 - `Point` 클래스 : `Point2i` 클래스와 같음, 가장 많이 사용됨, 2차원 좌표를 정수형으로 표현하는 자료형

Point_ 클래스

- Point_ 객체를 생성하고 초기화 하는 방법

```
Point pt1;                // pt1 = (0, 0)
pt1.x = 5; pt1.y = 10;    // pt1 = (5, 10)
Point pt2(10, 30);        // pt2 = (10, 30)
```

- Point_ 클래스는 다양한 연산자에 대해 연산자 함수가 정의가 되어 있음

```
// pt1 = [5, 10], pt2 = [10, 30]
Point pt3 = pt1 + pt2;      // pt3 = [15, 40]
Point pt4 = pt1 * 2;        // pt4 = [10, 20]
int d1 = pt1.dot(pt2);      // d1 = 350
bool b1 = (pt1 == pt2);     // b1 = false
```


Point_ 클래스

- std::cout과 << 연산자를 이용하여 Point_ 객체의 좌표를 쉽게 출력할 수 있음(<<연산자함수 정의되어 있음)

```
cout << "pt1: " << pt1 << endl;  
cout << "pt2: " << pt2 << endl;
```

- 실행결과 : [x, y] 형식으로 출력됨

```
pt1: [5, 10]  
pt2: [10, 30]
```

예제1 : 두점사이 거리 구하기

```
#include <opencv2/opencv.hpp>
```

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace cv;
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    Point pt1(10, 10);
```

```
    Point pt2(20, 20);
```

```
    double dist;
```

```
    dist = sqrt((pt1.x - pt2.x) * (pt1.x - pt2.x) + (pt1.y - pt2.y) * (pt1.y - pt2.y));
```

```
    cout << dist << endl;
```

```
    dist = norm(pt1 - pt2);
```

```
    cout << dist << endl;
```

```
    return 0;
```

```
}
```

Microsoft Visual Studio 디버그

14.1421

14.1421

Size_ 클래스

- 영상 또는 사각형의 크기를 표현할 때에는 사용하는 템플릿 클래스
- 사각형의 가로와 세로 크기를 나타내는 width와 height 멤버 변수를 가지고 있음

코드 3-2 간략화한 Size_ 클래스 정의와 이름 재정의

```

01  template<typename _Tp> class Size_
02  {
03  public:
04      Size_();
05      Size_(_Tp _width, _Tp _height);
06      Size_(const Size_& sz);
07
08      Size_& operator = (const Size_& sz);
09

```

템플릿 파라미터(매개 변수)

Size_ 클래스

코드 3-2 간략화한 Size_ 클래스 정의와 이름 재정의

```

10     _Tp area() const;
11     bool empty() const;
12
13     _Tp width, height; ← 폭(가로크기), 높이(세로크기)
14 };
15
16 typedef Size_<int>      Size2i;
17 typedef Size_<int64>    Size2l;
18 typedef Size_<float>    Size2f;
19 typedef Size_<double>   Size2d;
20 typedef Size2i          Size;

```

Size_ 클래스

- Size_ 클래스는 템플릿 클래스이기 때문에 객체 선언시 자료형을 명시해야 함

Size_<int> pt;

- 다양한 자료형에 대해 이름이 재정의되어 있음
 - Size2i 클래스: 사각형의 가로 및 세로 크기를 int형으로 표현
 - Size2f 클래스: 사각형의 가로 및 세로 크기를 float형으로 표현
 - Size 클래스: Size2i와 같음, 가장 많이 사용됨

Size_ 클래스

- 객체 생성 및 초기화

```
Size sz1, sz2(10, 20);           // sz1 = [0 x 0], sz2 = [10 x 20]
sz1.width = 5; sz1.height = 10;  // sz1 = [5 x 10]
```

- 다양한 연산자에 대해 연산자 함수를 제공함

```
// sz1 = [5 x 10], sz2 = [10 x 20]
Size sz3 = sz1 + sz2;    // sz3 = [15 x 30]
Size sz4 = sz1 * 2;      // sz4 = [10 x 20]
int area1 = sz4.area();  // area1 = 200
```

Size_ 클래스

- std::cout과 << 연산자를 이용하여 sz3과 sz4 객체의 정보를 화면에 출력(<< 연산자 함수 정의됨)

```
cout << "sz3: " << sz3 << endl;
```

```
cout << "sz4: " << sz4 << endl;
```

- 크기 정보가 [width×height] 형식으로 출력됨

```
sz3: [15 x 30]
```

```
sz4: [10 x 20]
```

예제2 : 면적 비교하기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Size s1(10, 10);
    Size s2(20, 20);
    int area1 = s1.area();
    int area2 = s2.area();
    if ( area1 > area2 ) cout << "s1의 면적이 크다" << endl;
    else if (area1 == area2) cout << "s1,s2 면적이 같다" << endl;
    else cout << "s2의 면적이 크다" << endl;

    return 0;
}
```

Microsoft Visual Studio 디

s2의 면적이 크다

Rect_ 클래스

- 사각형의 위치와 크기 정보를 표현할 때에는 사용하는 템플릿 클래스
- 사각형의 좌측 상단 점의 좌표를 나타내는 x, y 멤버 변수와 사각형의 가로 및 세로 크기를 나타내는 width, height 멤버 변수를 가지고 있음

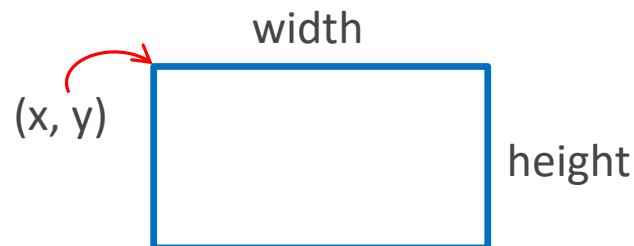
코드 3-3 간략화한 Rect_ 클래스 정의와 이름 재정의

```

01  template<typename _Tp> class Rect_
02  {
03  public:
04      Rect_();
05      Rect_(_Tp _x, _Tp _y, _Tp _width, _Tp _height);
06      Rect_(const Rect_& r);
07      Rect_(const Point_<_Tp>& org, const Size_<_Tp>& sz);
08      Rect_(const Point_<_Tp>& pt1, const Point_<_Tp>& pt2);
09

```

↑ 템플릿 매개변수



Rect_ 클래스

코드 3-3 간략화한 Rect_ 클래스 정의와 이름 재정의

```

10     Rect_& operator = ( const Rect_& r );
11
12     Point_<_Tp> tl() const;
13     Point_<_Tp> br() const;
14     Size_<_Tp> size() const;
15     _Tp area() const;
16     bool empty() const;
17     bool contains(const Point_<_Tp>& pt) const;
18
19     _Tp x, y, width, height;
20 };
21
22     typedef Rect_<int>      Rect2i;
23     typedef Rect_<float>    Rect2f;
24     typedef Rect_<double>  Rect2d;
25     typedef Rect2i          Rect;

```

← 위치,크기 멤버 변수

Rect_ 클래스

- 템플릿 클래스이므로 객체 선언시 자료형을 명시해야 함

`Rect_<int> rect;`

- 다양한 자료형에 대해 이름이 재정의되어 있음
 - `Rect2i` 클래스 : int형으로 사각형 정보를 표현
 - `Rect2f` 클래스 : float형으로 사각형 정보를 표현
 - `Rect` 클래스 : `Rect2i` 클래스와 같음, 가장 많이 사용됨,

Rect_ 클래스

- 객체 생성 및 초기화

```
Rect rc1; // rc1 = [0 x 0 from (0, 0)]
Rect rc2(10, 10, 60, 40); // rc2 = [60 x 40 from (10, 10)]
```

- Rect_ 클래스는 Size_ 또는 Point_ 클래스 객체와의 산술 연산자가 재정의 되어 있음

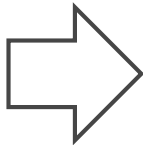
- Rect_ 객체 + Size_ 객체 : 사각형의 크기(width, height)만 변경됨
- Rect_ 객체 + Point_ 객체 : 사각형 위치(x,y)만 변경됨

```
// rc1 = [0 x 0 from (0, 0)], rc2 = [60 x 40 from (10, 10)]
Rect rc3 = rc1 + Size(50, 40); // rc3 = [50 x 40 from (0, 0)]
Rect rc4 = rc2 + Point(10, 10); // rc4 = [60 x 40 from (20, 20)]
```

C++고급 : 생성자의 직접호출

- 생성자를 직접 호출하면 임시객체를 생성하고 초기화한다
- 임시객체는 해당 수식(문장)에서만 사용되고 사라진다.

```
Size      sz(50,40);  
Point     pt(10,10);  
Rect rc3 = rc1 + sz;  
Rect rc4 = rc2 + pt;
```



```
Rect rc3 = rc1 + Size(50, 40);  
Rect rc4 = rc2 + Point(10, 10);
```

Rect_ 클래스

- & 연산 : 두 사각형이 교차하는 사각형 영역을 반환함
- | 연산 : 두 사각형을 모두 포함하는 최소 크기의 사각형을 반환함

```
// rc3 = [50 x 40 from (0, 0)], rc4 = [60 x 40 from (20, 20)]
Rect rc5 = rc3 & rc4;           // rc5 = [30 x 20 from (10, 10)]
Rect rc6 = rc3 | rc4;           // rc6 = [80 x 60 from (0, 0)]
```

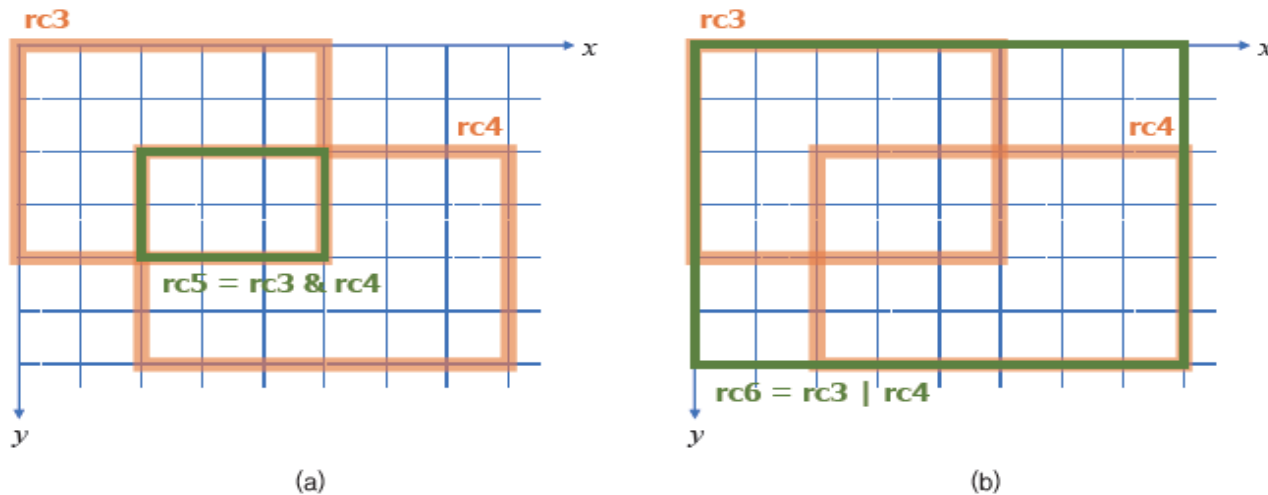


그림 3-1 Rect_ 클래스의 논리 연산 결과 예

Rect_ 클래스

- std::cout과 << 연산자를 이용하여 Rect_ 객체를 출력

```
cout << "rc5: " << rc5 << endl;  
cout << "rc6: " << rc6 << endl;
```

- 사각형 정보가 [width x height from (x, y)] 형식으로 출력됨

```
rc5: [30 x 20 from (20, 20)]  
rc6: [80 x 60 from (0, 0)]
```

예제3 : Rect 클래스 활용

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Rect r1(10, 10, 20, 20);
    Rect r2(20, 20, 30, 30);
    cout << "r1의 우측하단 좌표는" << r1.br() << endl;
    cout << "r2의 우측하단 좌표는" << r2.br() << endl;
    cout << "r1의 면적은" << r1.area() << endl;
    cout << "r2의 면적은" << r2.area() << endl;
    Rect r = r1 & r2;
    cout << "r1,r2의 겹치는 면적은" << r.area() << endl;

    return 0;
}
```

Microsoft Visual Studio 디버그 콘솔

r1의 우측하단 좌표는[30, 30]
r2의 우측하단 좌표는[50, 50]
r1의 면적은400
r2의 면적은900
r1,r2의 겹치는 면적은100

RotatedRect 클래스

- 생략

Range 클래스

- 범위 또는 구간을 표현하는 클래스임
- 범위의 시작과 끝을 나타내는 start와 end 멤버 변수를 가지고 있음

코드 3-5 간략화한 Range 클래스 정의

```

01  class Range
02  {
03  public:
04      Range();
05      Range(int _start, int _end);
06
07      int size() const;
08      bool empty() const;
09      static Range all();
10
11      int start, end;
12  };

```

Range 클래스

- Range 클래스는 start부터 end까지의 정수 단위 범위를 표현함
- 이때 start는 범위에 포함되고, end는 범위에 포함되지 않음
- 다음 예제 코드에서 r1은 0부터 9까지의 범위를 표현하고, 10은 포함하지 않음

```
Range r1(0, 10);
```

String 클래스

- C++ 표준 라이브러리(STL)에서 `std::string` 클래스를 이용하여 문자열을 저장하고 처리함
- OpenCV에서는 `cv::String` 클래스를 사용하여 문자열을 저장하고 처리할 수 있음
- OpenCV 4.0 버전부터는 자체적인 String 클래스 정의를 삭제하고, 대신 C++ 표준 라이브러리의 `std::string` 클래스를 String 클래스로 이름 재정의함

```
typedef std::string String;
```

String 클래스

- 덧셈 연산자를 이용하여 여러 문자열을 이어서 하나의 문자열을 만들 수 있음

```
String str1 = "Hello";  
String str2 = "world";  
String str3 = str1 + " " + str2; // str3 = "Hello world"
```

- 두 문자열 객체의 내용을 비교하기 위해서는 == 연산자를 사용할 수 있음

```
bool ret = (str2 == "WORLD");
```

String 클래스

```
String format(const char* fmt, ...);
```

- **fmt** 형식 문자열
- **...** 가변 인자
- **반환값** 지정된 형식으로 생성된 문자열

- 만약 특정한 형식의 문자열을 만들고 싶다면 OpenCV에서 제공하는 format() 함수를 사용할 수 있음
- C 언어에서 printf() 함수를 사용하듯이 함수의 인자를 설정하면 그에 해당하는 출력 문자열이 String 객체 형태로 반환함
- 실수 또는 정수형 변수에 저장된 값을 String 객체에 저장할 때 사용

```
double pi = 3.14159;
```

```
String str;
```

```
printf("원주율은 %f", pi);
```

```
str = format("원주율은 %f", pi);
```

화면

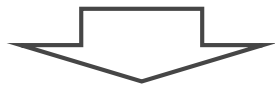
원주율은 3.14159

원주율은 3.14159

String 클래스

- 예를 들어 OpenCV 예제 프로그램에서 test01.bmp, test02.bmp, test03.bmp 세 개의 테스트 파일을 불러오고 싶을 때, 다음과 같은 코드를 이용하여 파일 이름에 해당하는 문자열을 생성할 수 있음

```
Mat imgs0, imgs1, imgs2;
imgs0 = imread("test01.bmp");
imgs1 = imread("test02.bmp");
imgs2 = imread("test03.bmp");
```



```
Mat imgs[3];
for (int i = 0; i < 3; i++) {
    String filename = format("test%02d.bmp", i + 1);
    imgs[i] = imread(filename);
}
```

예제4 : 숫자를 String 객체로 변환하기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    String str;

    for (int i = 0; i < 10; i++)
    {
        str = format("파일%02d.jpg", i);
        cout << str << endl;
    }
    return 0;
}
```

C:\> Microsoft Visual St

파일00.jpg
파일01.jpg
파일02.jpg
파일03.jpg
파일04.jpg
파일05.jpg
파일06.jpg
파일07.jpg
파일08.jpg
파일09.jpg

과제1

- 두 점의 좌표를 키보드로 입력 받아 화면에 출력하고 또, 두점 사이의 거리를 구하여 출력하라.

```
Microsoft Visual Studio 디버그 콘솔
첫번째 점의 x,y좌표를 입력:10 10
두번째 점의 x,y좌표를 입력:20 20
pt1:[10, 10]
pt2:[20, 20]
두점사이의 거리:14.1421
```

과제2

- 두 사각형의 크기를 키보드로 입력 받아 화면에 출력하고 또, 두 사각형의 면적을 각각 구하여 출력하라.

```
Microsoft Visual Studio 디버그 콘솔
첫번째 사각형의 폭,높이 입력:10 10
두번째 사각형의 폭,높이 입력:20 20
s1:[10 x 10]
s2:[20 x 20]
s1의 면적100
s2의 면적400
```

과제3

- Rect 클래스의 멤버함수 contains를 이용하여 아래 코드의 점 p1이 사각형 r1의 영역 내부에 존재하는지 판단하는 코드를 작성하시오. contains함수 사용법은 opencv사이트 접속하여 매뉴얼을 검색하라.

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
Using namespace std;
Int main()
{
    Rect r1(10, 10, 20, 20);
    Point p1(15, 15);

    // 사각형 r1안에 점 p1이 존재하는지 체크하는 코드
    // 내부에 존재 또는 외부에 존재로 결과 출력할 것

    return 0;
}
```

과제4

- 사각형 정보와 점의 좌표를 입력 받아 점이 사각형 내부에 있는지 외부에 있는지 결과를 출력하는 코드를 작성 하시오.
- Point_ 클래스의 멤버함수 inside함수를 이용하여 판단 하시오.
opencv.org사이트의 도움말을 이용할 것

사각형의 좌측상단의 좌표를 입력하시오(x,y): 10 20 <엔터>
사각형의 폭,높이를 입력하시오(width, height): 50 30 <엔터>
점 P의 좌표를 입력하시오: 50 40 <엔터>
점 P는 사각형 안에 있다.

과제5

- 예제4를 아래 결과가 나오도록 수정 하시오.

실행결과1.bmp
실행결과3.bmp
실행결과5.bmp
실행결과7.bmp
실행결과9.bmp
실행결과11.bmp
실행결과13.bmp
실행결과15.bmp

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.